

Multiprocessadors (MP): LAB sessió 2

1 Cache amb escriptura immediata

En aquesta pràctica es dissenyarà una cache bloquejant, amb escriptura immediata i sense assignació de contenidor en cas de fallada en escriptura.

En la Figura 1 es mostren les interfícies de la cache amb el processador i la memòria. La funcionalitat dels senyals es descriu a la taula de la Figura 2. En el processador es distingeix la part productora (P) i la part consumidora (C). En l'esquema, el bus es mostra dues vegades amb l'objectiu que els senyals, amb alguna excepció, flueixin d'esquerra a dreta. Els senyals de les interfícies que es connecten amb el bus s'anomenen en aquest amb el prefix `bus_`.

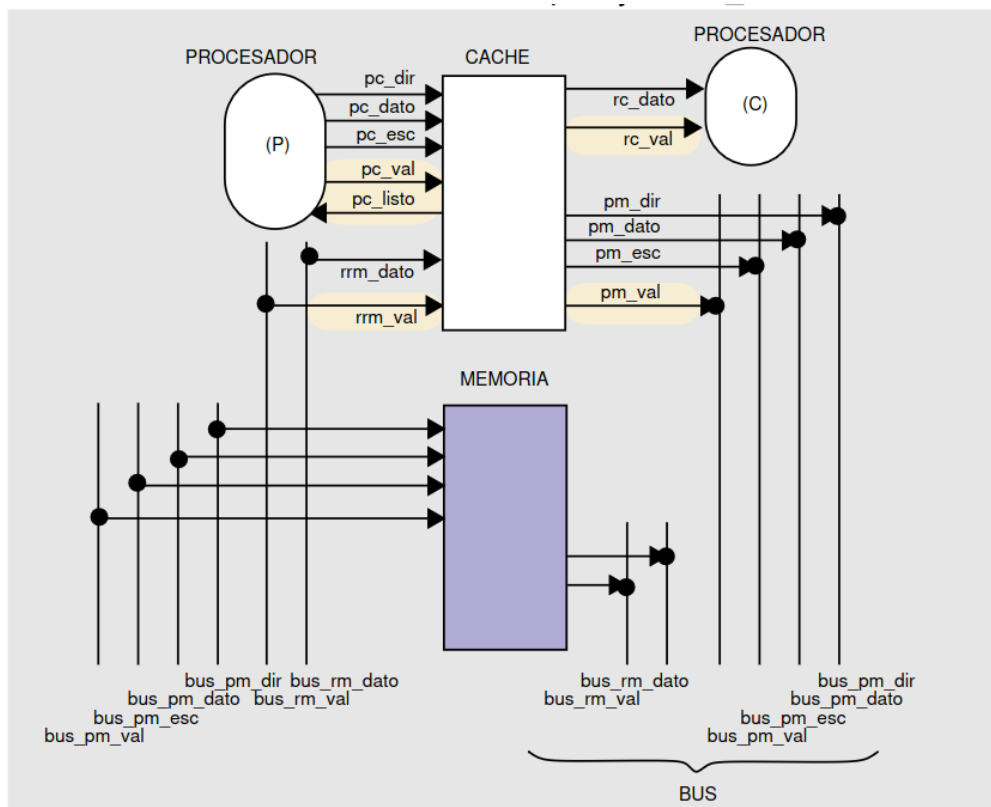


Figura 1: Interfícies entre processador, cache i memòria.

Interfaces procesador / cache			Interfaces cache / (bus) memoria		
		Descripción de las señales			Descripción de las señales
petición	información	pc_dir	dirección: dirección generada por el procesador	pm_dir	dirección: dirección de acceso a memoria
		pc_dato	dato: dato en una instrucción store	pm_dato	dato: dato en un acceso de escritura
		pc_esc	escritura: indicación de escritura	pm_esc	escritura: indicación de escritura
		pc_val	válido: indicación de acceso	pm_val	válido: indicación de acceso
		pc_listo	listo: cache desocupada (o libre)		
respu.	pr.				
	inf.	rc_dato	dato: respuesta de la cache en una instrucción load	rm_dato	dato: respuesta de la memoria en una petición de bloque
		rc_val	válido: indicación de respuesta	rm_val	válido: indicación de respuesta

Figura 2: Descripció dels senyals de les interfícies entre processador, cache i memòria.

Control de flux. El control de flux entre el processador (P) i la cache utilitza un protocol llest/vàlid (`pc_listo/pc_val`). El control de flux entre la cache i el processador (C) utilitza un protocol degenerat (`rc_val`), més simple, ja que suposem que el processador sempre està disposat a acceptar la dada subministrada per la cache. Pel que fa al control de flux entre la cache i la memòria (`pm_val`) i viceversa (`rm_val`), suposem que tant la cache com la memòria sempre estan disposades a acceptar la petició o la dada.

1.1 Organització de la cache

Com és habitual, una adreça de memòria referencia una posició que emmagatzema un byte. Per simplificar el disseny, tots els accessos a memòria generats pel processador són a byte.

La cache que es dissenya és de correspondència directa i té 8 contenidors. En un contenidor es distingeixen els camps: etiqueta, estat i bloc de dades. Només els blocs emmagatzemats en cache tenen un maquinari que representa el seu estat.

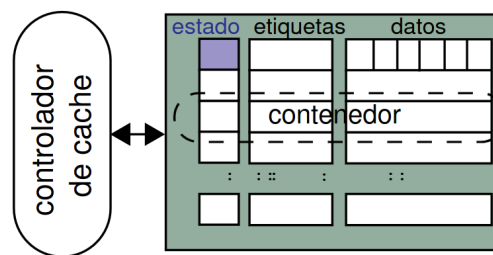


Figura 3: Memòria cache i camps en un contenidor de cache.

Per centrar-nos en les accions que ha de realitzar el controlador de cache quan rep peticions del processador, suposarem que la mida de bloc és igual a la granularitat d'accés del processador, en aquest cas 1 byte. El controlador de cache actualitza l'estat dels blocs en resposta a esdeveniments del processador i genera accessos a memòria.

1.2 Esdeveniments del processador i peticions d'accés

En la taula de la Figura 4 es descriuen els esdeveniments del processador i les accions del controlador de cache. Els esdeveniments principals són la lectura del processador (LPr) i l'escriptura del processador (EPr). Les accions del controlador inclouen la petició de bloc (Pt), l'actualització de memòria (PtE) i el reemplaçament d'un bloc (CcRe).

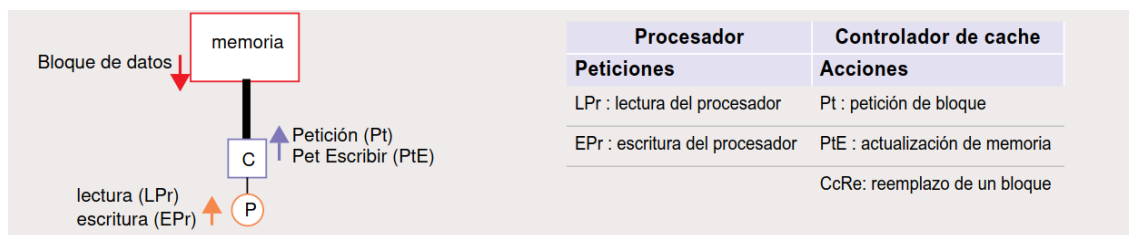


Figura 4: Esdeveniments del processador i accions del controlador de cache.

1.3 Diagrama d'estats i transicions d'un bloc en cache

En la Figura 5 es mostren els dos estats d'un bloc en cache i les transicions entre ells. L'estat d'un bloc només és explícit quan està emmagatzemat en cache.

L'estat invàlid (I) indica que el contenidor no emmagatzema un bloc vàlid. Un bloc que no està emmagatzemat en un contenidor de cache es diu que no està present (fallada de cache) i, per tant, està en estat invàlid.

L'estat vàlid (V) indica que el contenidor emmagatzema un bloc vàlid. Una operació de reemplaçament del controlador de cache selecciona un bloc i habilita el contenidor que l'emmagatzema per a un altre ús; el bloc s'invalida i s'indica en el contenidor. En el cas que ens ocupa, escriptura immediata, aquesta operació no té cap acció associada, ja que la memòria sempre està actualitzada.

Si en una operació de lectura (LPr) es produeix encert en cache, se subministra la dada. En cas contrari, es genera una petició de lectura a memòria del bloc on està situada la dada accedida (Pt).

En una operació d'escriptura es genera una petició per actualitzar memòria (PtE) i, si és un encert, s'actualitza el camp de dades en cache. La granularitat d'una actualització de memòria, en aquest disseny, és un byte.

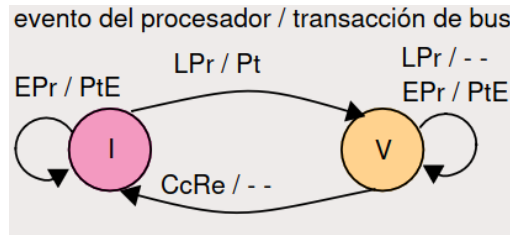


Figura 5: Diagrama d'estats i transicions d'un bloc en un contenidor de cache.

2 Camí de dades i controlador de cache

En la Figura 6 es mostren les interfícies del camí de dades de la cache, el controlador de cache, el processador i la memòria. Els senyals entre el controlador de cache i el mòdul de camps es descriuen posteriorment. Associat al camí de dades es distingeix un multiplexor, muxE, que s'utilitza per actualitzar el mòdul de camps en fallades de lectura i encerts d'escriptura.

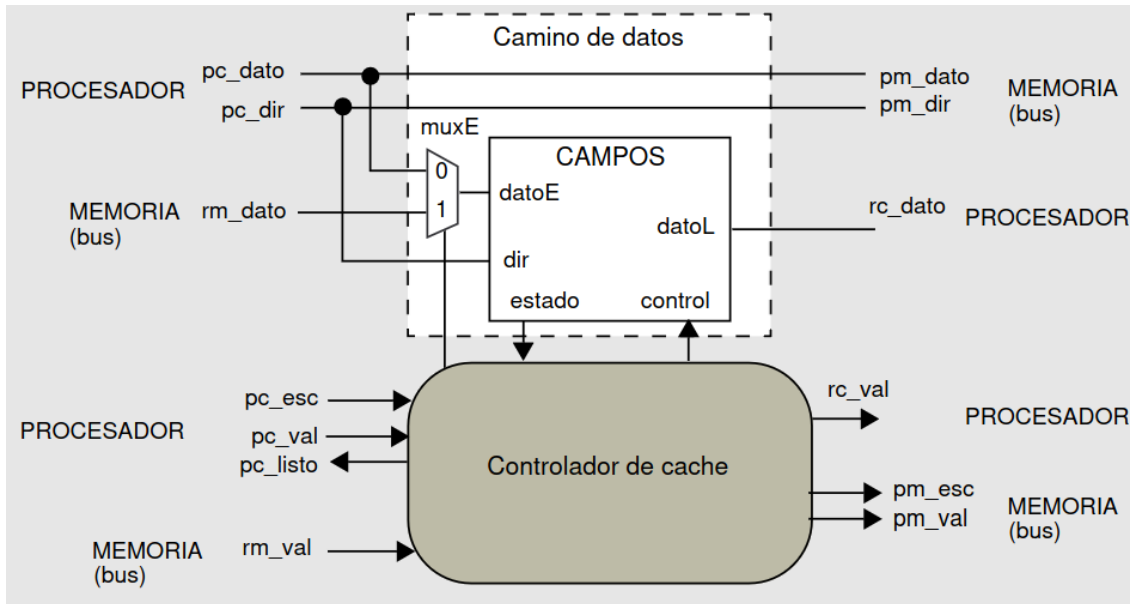


Figura 6: Memòria cache: mòdul de camps i controlador de cache.

2.1 Elements en el mòdul camps

En la Figura 7 es mostren els elements del mòdul de camps. Es distingeixen tres elements d'emmagatzematge, utilitzats per implementar els tres camps dels contenidors de cache.

Cada camp s'implementa amb un element d'emmagatzematge que té una entrada **dir** per indicar l'entrada accedida, és a dir, el camp índex o conjunt d'una adreça de memòria. Una altra entrada és

el camí d'accés per a escriptures (DE), amb un nombre de bits que depèn del camp corresponent de la cache. Les altres entrades (*acc* i *pe*) s'utilitzen per accedir al camp i escriure-hi, si escau.

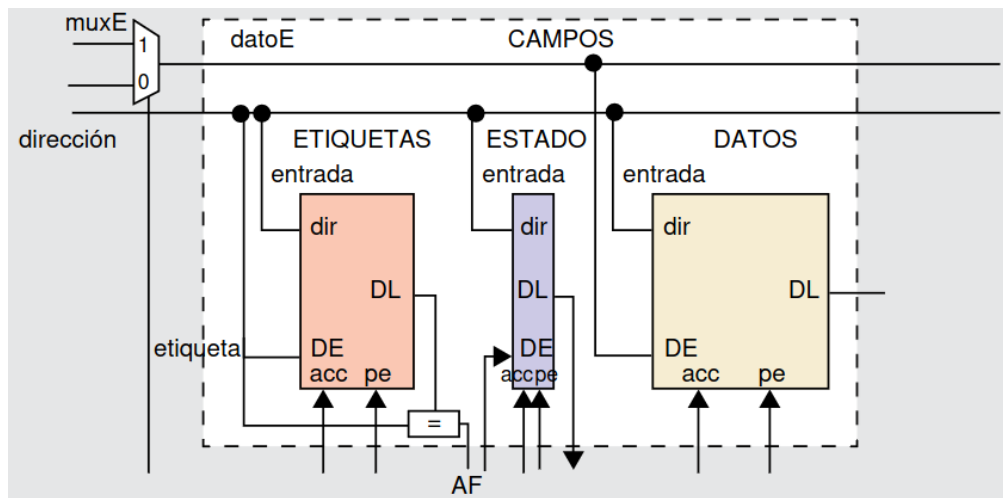


Figura 7: Elements d'emmagatzematge i circuits addicionals en el mòdul camps.

Señales	Número de bits
dirección	16 bits
dir (entrada)	3 bits
DE y DL en Etiquetas	13 bits
DE y DL en Estado	1 bit
DE y DL en Datos	8 bits
acc y pe	1 bit

Figura 8: Mida de les entrades i sortides dels camps de la cache.

Els paràmetres de cadascun dels camps s'indiquen a la taula de la Figura 8. El nombre de conjunts de la cache és 8. Els elements de memorització utilitzats són síncrons: tenen com a entrada el senyal de rellotge. Tant la lectura com l'escriptura estan sincronitzades amb el flanc ascendent del senyal de rellotge. En aquestes condicions només hi ha un port d'accés a cada camp de la cache. Un camp amb el senyal *acc* activat s'està llegint i pot escriure's si està activat el permís d'escriptura corresponent. En el cicle en què s'escriu, el valor que es llegeix no ha de correspondre necessàriament amb el valor escrit; el valor llegit no està especificat.

2.2 Controlador de cache

En la Figura 9 es mostren les interfícies entre el controlador de cache i els camps de la cache, a més del processador i la memòria. En el cas dels camps de cache s'identifiquen els senyals com d'estat o de control.

Descripció textual del controlador de cache. Quan el senyal *pc_val* (petició) s'activa, el controlador de cache utilitza el senyal *AF* per determinar si s'ha produït encert o fallada. Els drets d'accés es determinen utilitzant el senyal *AF* i el valor llegit del camp estat (*est*). El senyal *pc_esc* indica si l'accés és de tipus lectura o escriptura.

Si la instrucció és un *load* i es detecta encert, la dada llegida del camp de dades se subministra al processador. El processador (C) espera l'activació del senyal *rc_val* per capturar la dada. El processador (P) determina quan pot efectuar una nova petició observant l'activació del senyal *pc_listo*.

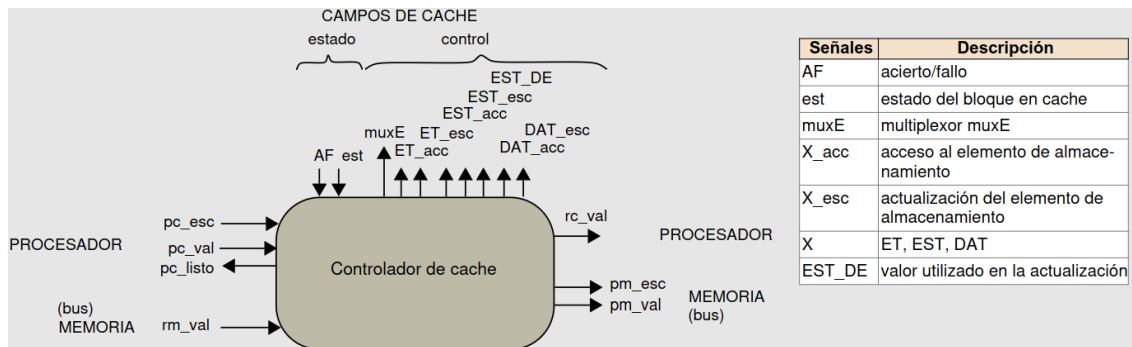


Figura 9: Interfícies del controlador de cache amb la cache, el processador i la memòria.

En el cas d'una escriptura o d'una fallada de lectura és necessari interactuar amb memòria, cosa que es realitza mitjançant el senyal de petició `pm_val`. El tipus de petició a memòria (lectura o escriptura) s'indica mitjançant el senyal `pm_esc`. L'activació de `rm_val` indica que s'ha servit l'accés a memòria. En una lectura de memòria s'actualitza la cache amb el bloc, se subministra la dada al processador i s'activen `pc_listo` i `rc_val`. En una escriptura a memòria, si hi ha encert en cache, s'actualitza el camp de dades de cache en rebre la confirmació de memòria. Tant si l'escriptura ha estat encert com fallada, el servei ha finalitzat i s'activa `pc_listo`.

```

while (true) {
    while (pc_val = NO) { };
    if (AF = acierto and pc_esc = lectura and estado = válido) then      acierto de lectura
        pc_listo = SI; rc_val = SI;
    else
        pm_val = petición a memoria;
        pm_esc = pc_esc;
        while (rm_val = NO) { };
        if (pc_esc = lectura) then                                       fallo de lectura
            muxE = memoria;
            DAT_acc, DAT_esc = escribir campo datos;
            EST_acc, EST_esc = escribir campo estado;
            EST_DAT = válido;
            ET_acc, ET_esc = escribir campo etiquetas; rc_val = si;
        else if (AF = acierto and pc_esc = escriptura) then            acierto de escriptura
            muxE = procesador;
            DAT_acc, DAT_esc = escribir campo datos;
        end if
        pc_listo = si;
    end if
}

```

Figura 10: Descripció textual en pseudocodi de les accions del controlador.

Fases en l'accés a cache. En l'accés a cache distingim diverses fases. Les dues primeres són comunes a tots els tipus d'accés considerats: a) accés als camps d'etiquetes i estat (ET, EST), i b) comparació d'etiquetes i determinació dels drets d'accés utilitzant el senyal d'estat del bloc (CMPET). La Figura 11 mostra la seqüència de fases per a cada cas.

Les fases següents depenen de si hi ha encert o fallada i del tipus d'accés. En una lectura amb drets d'accés, s'accedeix al camp de dades (LEC). Si no es disposa de drets d'accés, es fa una petició de bloc a memòria (MEM), posteriorment s'actualitza la cache amb el bloc rebut (ESB) i finalment s'accedeix al camp de dades per llegir la dada (LEC). En una escriptura sempre s'accedeix a memòria (MEM); quan es disposa de drets d'accés, també s'actualitza la cache (ESP).

	Fases				
acceso / DA	a)	b)	c)	d)	e)
Load / si	ET, EST	COMPET	LEC		
Load / no	ET, EST	COMPET	MEM	ESB	LEC
Store / si	ET, EST	COMPET	MEM	ESP	
Store / no	ET, EST	COMPET	MEM		

Figura 11: Fases en una transacció de memòria. Drets d'accés: DA (AF i est).

Autòmat del controlador de cache. En el controlador de cache s'identifica un conjunt d'estats que permeten seqüenciar les fases. La Figura 12 mostra tots els estats del controlador de cache. L'estat INI s'afegeix per comprovar el funcionament del camí de dades de la cache i del seu control. Aquesta petició de inici permet escriure en els diferents camps d'un contenidor de cache, independentment de l'estat. Un cop finalitzada, el contenidor de cache conté un bloc vàlid i posteriorment pot efectuar-se una operació de lectura que encerta en cache. Aquesta nova petició, només pot efectuar-se després d'inicialitzar (posar a zero) el controlador.

ESTADOS	Descripción
DES0 y/ o DES	Estado inicial después de una puesta a cero (Figura 14 y Figura 16). DES estado de espera (Figura 16)
INI	Nada
ESCINI	Actualización de todos los campos, asociados al contenedor referenciado, con la información suministrada en la transacción.
HECHOE	Acceso de escritura finalizado
HECHOL	Acceso de lectura finalizado
COMPET	Lectura del campo etiqueta y campo estado. Comparación de etiquetas
LEC	Lectura del campo datos
PML	Inicio del acceso a memoria para leer un bloque
ESPL	Espera a que la memoria suministre el bloque
ESB	Actualización de todos los campos, del contenedor asociado, con el bloque leído de memoria
PMEA	Inicio del acceso a memoria para escribir un dato cuando la cache contiene el bloque
ESPEA	Espera a que la memoria confirme la escritura del dato cuando la cache contiene el bloque
ESCP	Actualización del campo de datos debido a una escritura que acierta en cache
PMEF	Inicio del acceso a memoria para escribir un dato cuando la cache no contiene el bloque
ESPEF	Espera a que la memoria confirme la escritura del dato cuando la cache no contiene el bloque

Figura 12: Estats del controlador de cache.

Graf de transicions entre estats. En la Figura 14 es mostra el graf de transicions entre estats del controlador de cache.

acceso / DA	Fases y transiciones entre estados						
Load / si	DES	CMPET	LEC	HECHOL			
fases		ET, EST, CMPET	LEC				
Load / no	DES	CMPET	PML	ESPL	ESB	LEC	HECHOL
fases		ET, EST, CMPET	MEM	MEM	ESB	LEC	
Store / si	DES	CMPET	PMEA	ESPEA	ESCP	HECHOE	
fases		ET, EST, CMPET	MEM	MEM	ESP		
Store / no	DES	CMPET	PMEF	ESPEF	HECHOE		
fases		ET, EST, CMPET	MEM	MEM			

Figura 13: Transicions entre estats en funció del tipus d'accés i dels drets d'accés.

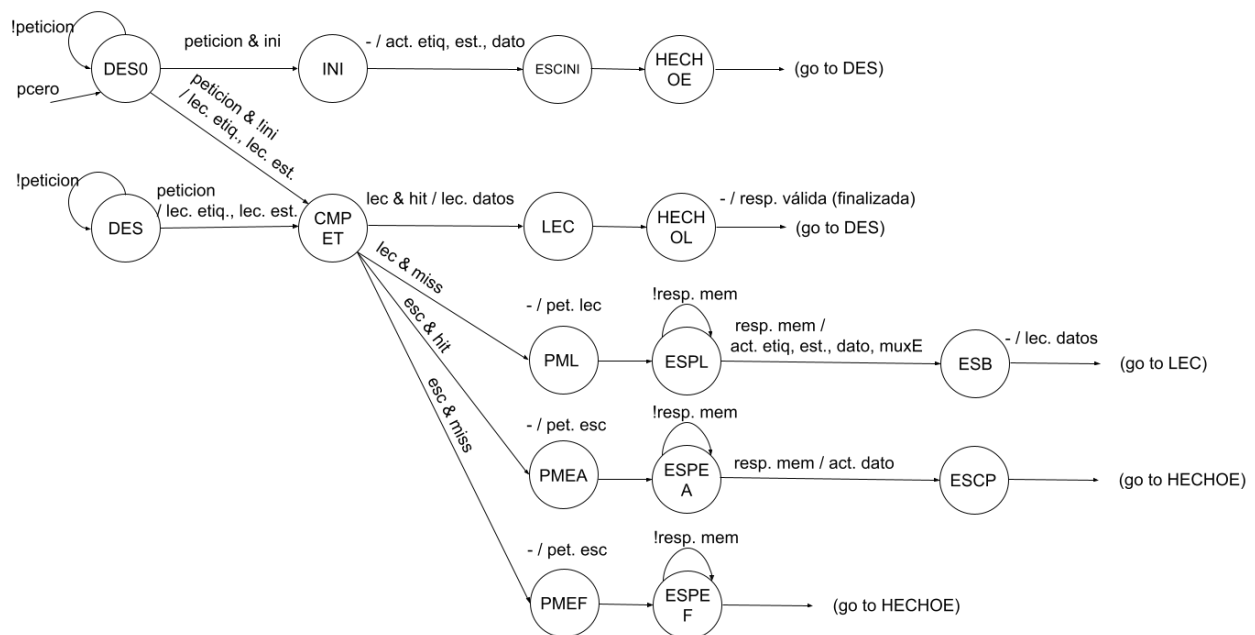


Figura 14: Graf de transicions entre estats del controlador de cache.

2.3 Accés a memòria

L'accés a memòria és multicicle. Els rectangles negres de la Figura 15 representen registres de desacoblament o registres interns del mòdul bàsic d'emmagatzematge. En el cicle posterior a la detecció d'una escriptura o d'una fallada de lectura, es transmet pel bus la informació oportuna (**bus_I**, bus Ida). La memòria està ocupada durant dos cicles. En el cicle posterior s'ocupa el bus (**bus_V**, bus Vuelta) i en el següent el controlador de cache detecta la finalització de l'accés a memòria.

En la Figura 16 es mostren els cicles per a cadascuna de les transaccions. En l'última fila es mostra l'ocupació del bus (anada i tornada), de la memòria i del controlador de cache en començar i finalitzar una transacció.

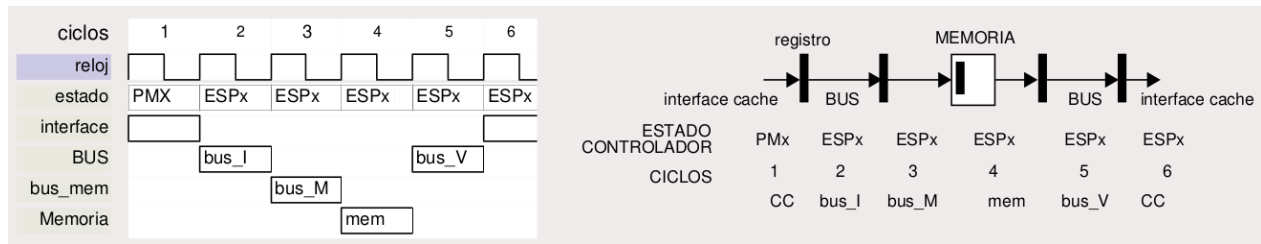


Figura 15: Detall d'una transacció en el bus.

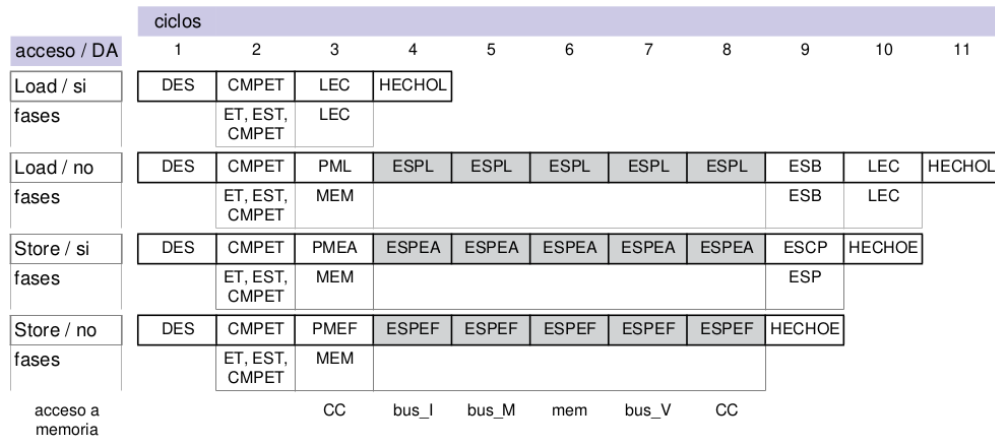


Figura 16: Cicles en funció del tipus de transacció. Drets d'accés: DA (AF i est).

3 Projectes

Aquesta pràctica inclou diversos projectes. Tots els projectes es refereixen al controlador de cache (CC), amb alguna modificació necessària en el camí de dades.

El treball es planteja de forma incremental. En el primer projecte el disseny del CC se centra en la funcionalitat: el seqüenciamnt i la temporalitat de les accions necessàries en funció del tipus d'accés, sense expectatives de rendiment. Els projectes següents tenen com a objectiu millorar el rendiment eliminant estats del CC per reduir la latència de l'accés a la jerarquia de memòria.

Els fitxers comuns a tots els projectes tenen com a arrel el directori **proyecto_1**. Els altres projectes tenen la mateixa estructura de directoris, però només inclouen els directoris i fitxers específics del projecte. Hi ha excepcions, com alguns *packages* que especifiquen procediments. Encara que els fitxers siguin idèntics en diversos projectes, es repliquen en cada projecte per independitzar-ne les proves.

3.1 Projecte 1: Disseny del controlador de cache

En el projecte 1 es requereix la implementació del controlador de memòria cau descrit a la Figura 14. Primerament, haureu de construir la taula de transicions, que conté la lògica de canvi d'estat i de les sortides. Finalment, haureu de programar en VHDL la lògica de canvi d'estat i la de sortides al fitxer `controlador.vhd` ubicat al directori `cache_con_interface_bus/cache_con_interface_proc/cache/controlador/CODIGO`. En aquest directori també s'inclou un fitxer VHDL amb l'especificació de funcions i procediments d'utilitat. El fitxer que conté la declaració del conjunt d'estats s'anomena `controlador_pkg.vhd` i és en el directori `tipos_constantes_pkg`. Analitzeu en detall les funcions i procediments i utilitzeu-los en la codificació del controlador de cache. A l'Annex 1, es detalla l'equivalència entre els noms de les variables de les figures mostrades a l'enunciat i les utilitzades en el codi VHDL.

Recomanació. És recomanable codificar i comprovar el funcionament de forma incremental. Per exemple, començar amb les transicions entre estats de la transacció d'inici i continuar amb les transicions d'una instrucció *load* que encerta en cache. Amb aquestes dues transaccions es comprova un funcionament bàsic del camí de dades de la cache i de la part implementada del controlador.

Programa de prova. Per comprovar el funcionament se subministra un fitxer amb el programa de prova i un altre amb procediments d'utilitat per emetre peticions. El programa de prova disposa, entre altres, d'un procés productor i un procés consumidor. Al productor s'especifica la seqüència de peticions les transaccions de les quals s'utilitzen per comprovar el sistema. El consumidor comprova el valor llegit en un accés de lectura. Un cop executat, si el controlador de memòria cau és correcte, haurà de produir els mateixos resultats que els de la carpeta ENSAMBLADO/RESULTADOS/RESULTADOS_VALIDOS/.

Exercicis. Realitza els següents treballs de l'entregable: Treball 1 i 2.

3.2 Projecte 2: Reducció del nombre d'estats

L'objectiu és reduir la latència en diverses transaccions. Per fer-ho s'eliminen alguns estats del controlador de cache: a) PMX (PML, PMEA, PMEF), b) HECHOX (HECHOE, HECHOL) i c) l'estat LEC en una transacció *load* que encerta en cache. La supressió d'un estat requereix efectuar les accions que s'hi efectuaven en un estat previ de la seqüència de transicions. En el cas de l'eliminació de LEC en un encert de lectura, l'accés al camp DAT és especulatiu: es llegeix el camp sense modificar informació de la cache i s'utilitza quan es disposa dels drets d'accés. Primerament, haureu de dissenyar la nova taula de transicions. Després, implementeu el disseny RTL en VHDL del nou controlador.

En la Figura 17 es mostren els cicles per a cada transacció un cop eliminats tots els estats. En l'última fila es mostra l'ocupació del bus, de la memòria i del CC al final d'una transacció.

		ciclos								
acceso / DA		1	2	3	4	5	6	7	8	9
Load / si		DES	CMPET							
fases			ET, EST, CMPET, LEC							
Load / no		DES	CMPET	ESPL	ESPL	ESPL	ESPL	ESPL	ESB	LEC
fases			ET, EST, CMPET	MEM					ESB	LEC
Store / si		DES	CMPET	ESPEA	ESPEA	ESPEA	ESPEA	ESPEA	ESCP	
fases			ET, EST, CMPET	MEM					ESP	
Store / no		DES	CMPET	ESPEF	ESPEF	ESPEF	ESPEF	ESPEF		
fases			ET, EST, CMPET	MEM						
acceso a memoria				bus_I	bus_M	mem	bus_V	CC		

Figura 17: Reducció del nombre d'estats. Cicles en funció del tipus de transacció.

Aquest projecte es troba en el directori `proyecto_2`. L'estructura de directoris és mimètica a la del `proyecto_1`. El codi VHDL de partida per al controlador de cache és el dissenyat prèviament. Copieu el cos de l'arquitectura del fitxer `controlador.vhd` del `proyecto_1` en el fitxer mimètic d'aquest projecte. En els directoris mimètics inclosos en `proyecto_2` hi ha els fitxers que contenen l'especificació de les funcions i procediments utilitzats pel controlador (`procedimiento_controlador_pkg.vhd`) i els possibles estats del controlador (`controlador_pkg.vhd`).

Recomanació. En els treballs següents es recomana efectuar una eliminació incremental dels estats, estat a estat, i comprovar-ne el funcionament. Per a cada pas incremental, construïu la

taula de transicions entre estats, on també s'especifiqui la lògica de sortida. Modifiqueu el codi del controlador de cache que heu copiat. Utilitzeu programes de prova per comprovar el funcionament de forma exhaustiva.

Programa de prova. Per comprovar el funcionament se subministra un fitxer amb el programa de prova. De la mateixa manera que al projecte 1, si el controlador de memòria cau és correcte, haurà de produir els mateixos resultats que els de la carpeta ENSAMBLADO/RESULTADOS/RESULTADOS_VALIDOS/.

Exercicis. Realitza els següents treballs de l'entregable: Treball 3 i 4.

3.3 Projecte 3. Eliminació completa de l'estat LEC

L'objectiu és reduir la latència en una fallada de lectura de cache. Per fer-ho cal eliminar l'estat LEC. En la Figura 18 es mostren els cicles per a cadascuna de les transaccions. En l'última fila es mostra l'ocupació del bus, de la memòria i del CC al final d'una transacció.

Per eliminar LEC, les accions d'actualització del contenidor de cache i subministrament de la dada al consumidor s'han d'efectuar en paral·lel. En conseqüència, cal modificar el camí de dades a més del controlador de cache.

		ciclos							
acceso / DA		1	2	3	4	5	6	7	8
Load / si	DES	CMPET							
fases			ET, EST, CMPET LEC						
Load / no	DES	CMPET	ESPL	ESPL	ESPL	ESPL	ESPL	ESPL	ESB
fases			ET, EST, CMPET	MEM					ESB, LEC
Store / si	DES	CMPET	ESPEA	ESPEA	ESPEA	ESPEA	ESPEA	ESPEA	ESCP
fases			ET, EST, CMPET	MEM					ESP
Store / no	DES	CMPET	ESPEF	ESPEF	ESPEF	ESPEF	ESPEF	ESPEF	
fases			ET, EST, CMPET	MEM					
acceso a memoria				bus_I	bus_M	mem	bus_V	CC	

Figura 18: Eliminació completa de l'estat LEC. Cicles en funció del tipus de transacció.

Modificació del camí de dades. En la Figura 19 es mostra la modificació del camí de dades de la cache. Cal afegir un multiplexor, *muxL*, i el seu control. Aquest multiplexor permet fer un "bypass" del contingut llegit a memòria. D'aquesta manera, mentre s'actualitza el valor llegit de memòria a la memòria cau, també s'envia al consumidor.

Dissenyau la nova taula de transicions, que ara afegeix la senyal de sortida del controlador *muxL*, i el codi RTL del nou controlador. Aquest projecte es troba en el directori **proyecto_3**. L'estructura de directoris és mimètica a la del **proyecto_1**. El codi VHDL de partida per al controlador de cache és el dissenyat en el **proyecto_2**. Copieu el cos de l'arquitectura del fitxer **controlador.vhd** del **proyecto_2** en el fitxer mimètic d'aquest projecte. En els directoris mimètics inclosos en **proyecto_3** hi ha els fitxers de funcions i procediments del controlador (**procedimiento_controlador_pkg.vhd**) i els possibles estats (**controlador_pkg.vhd**).

Programa de prova. Per comprovar el funcionament se subministra un fitxer amb el programa de prova. De la mateixa manera que als projectes previs, si el controlador de memòria cau és correcte,

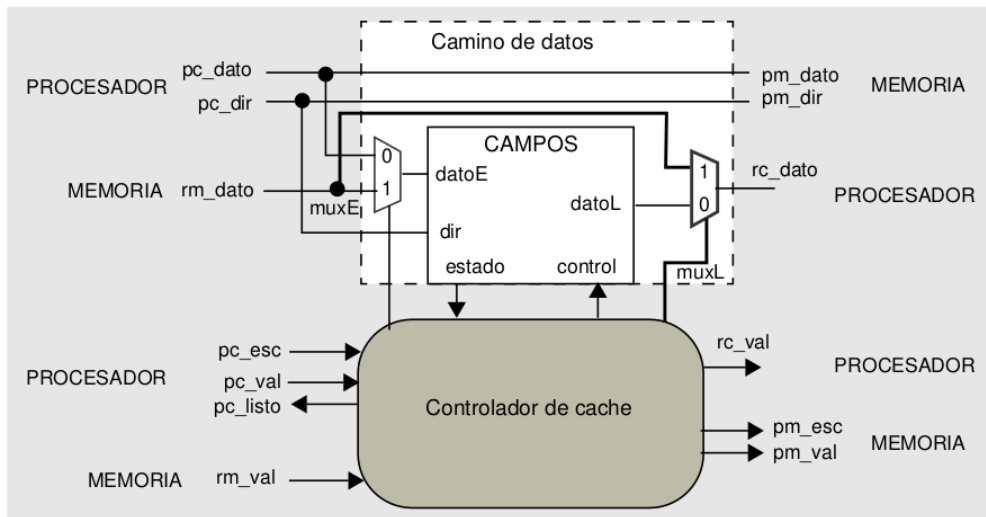


Figura 19: Camí de dades de la cache on concurrentment s'actualitza la cache i se subministra la dada al processador.

haurà de produir els mateixos resultats que els de la carpeta ENSAMBLADO/RESULTADOS/RESULTADOS_VALIDOS/.

Exercicis. Realitza els següents treballs de l'entregable: Treball 5 i 6.

Appendices

Apèndix 1: Senyals VHDL

En la section .0 es mostra un esquema de la interfície del controlador de cache utilitzada en el codi subministrat. El nom de les senyals en el codi VHDL és el que es mostra **SENSE** la negreta. Les estructures dels tipus de dades de les senyals són aquelles que comencen per *tp_*. A la section .0 es detalla la funcionalitat de cada senyal.

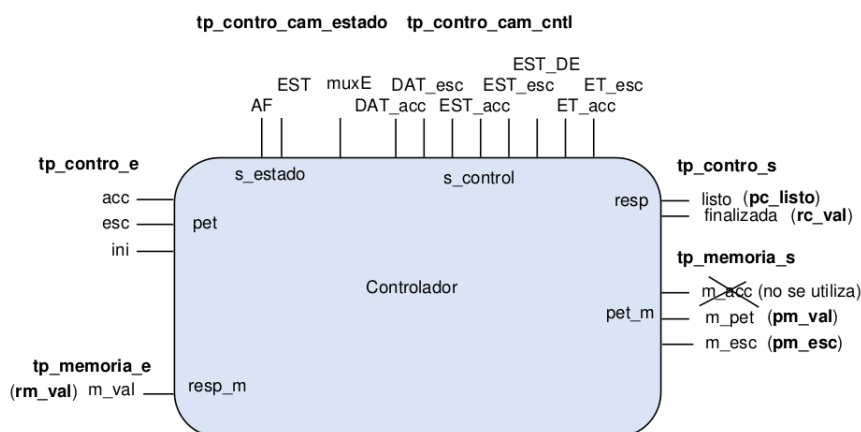


Figura 20: Señales VHDL del controlador de cache.

Código	tabla		
resp	resp.listo	pc_listo	cache lista u ocupada
	resp.finalizada	rc_val	en la salida de la cache hay un dato válido
s-control	s_control.ET_acc	ET_acc	acceso al campo etiquetas
	s_control.ET_esc	ET_esc	actualización del campo etiquetas
	s_control.EST_acc	EST_acc	acceso al campo estado
	s_control.EST_esc	EST_esc	actualización del campo estado
	s_control.EST_DE	EST_DE	valor con el cual se actualiza el campo estado
	s_control.DAT_acc	DAT_acc	acceso al campo etiquetas
	s_control.DAT_esc	DAT_esc	actualización del campo etiquetas
	s_control.muxE	muxE	selección de la entrada del multiplexor. Actualización del campo datos con el dato transportado por el bus (muxE = 1)
pet_m	pet_m.m_pet	pm_val	acceso a memoria
	pet_m.m_esc	pm_esc	actualización de memoria
pet	pet.acc		petición de lectura o escritura
	pet_esc		la petición es de escritura
	pet.ini		petición de inicio
resp	resp_m.m_val		respuesta de memoria
s_estado	s_estado.AF		resultado de la comparación de etiquetas
	s_estado.EST		estado leído del campo EST

Figura 21: Descripción de las señales VHDL del controlador de cache.